REAL-TIME VIDEO PIPELINE, SKELETAL DATA PROCESSING AND BIOMECHANICAL JUMP ANALYSIS FOR ACCESSIBLE ATHLETIC PERFORMANCE TRACKING

Y.Manulakshan

(IT22283962)

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science (Hons) in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

May 2026

DECLARATION OF CANDIDATE AND SUPERVISOR

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature: _____

Date: _____

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor: _____

Date: _____

DEDICATION

I dedicate this dissertation to my beloved family, whose unwavering support, encouragement, and sacrifices have been the foundation of my academic journey. To my parents, who instilled in me the value of education and perseverance, and to my siblings, who have always believed in my potential, this achievement belongs to you as much as it does to me. Their constant motivation during long working hours, late-night debugging sessions, and challenging research phases has been the quiet engine behind every milestone reached in this work.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mr. Pradeep K.W. Abeygunawardhana, and my co-supervisor, Ms. Akshi De Silva, for their invaluable guidance, continuous support, and encouragement throughout this research project. Their expertise in computer vision, signal processing, and applied machine learning shaped the technical direction of the video pipeline and biomechanical analysis modules described in this dissertation. They provided essential feedback during the design of the calibration logic, suggested improvements to the data persistence layer, and offered constant motivation during the more demanding phases of implementation and testing.

I am deeply grateful to the Sri Lanka Institute of Information Technology for providing the necessary resources, laboratory facilities, and academic environment that enabled me to conduct this research. The Faculty of Computing has been a supportive community throughout my undergraduate studies, and the department staff have always been helpful with administrative matters and technical assistance. Access to computing resources and a stable testing environment was particularly important for the latency benchmarking work reported in Chapter 4.

I would also like to thank my fellow NeuroMotion team members: Mathumitha Gnanasekaran (IT21379338), Leashan Marino Sylvester (IT22311368), Manulakshan Yogarajah (IT22283962), and Giritharan Jude Jawakker (IT22330864). Our collaborative efforts and shared dedication made this project possible. Each member contributed unique expertise that enriched the overall system, and the integration points between my video pipeline component and the downstream recommendation, voice feedback, and posture-correction modules required close cooperation and patient cross-team debugging. The team spirit and mutual support throughout the project timeline made the experience both productive and enjoyable.

Special thanks to the more than twenty volunteers who participated in the data collection process, contributing their time and effort to record exercise videos under varied lighting and camera-angle conditions. Their footage was essential for tuning the smoothing coefficient, validating the calibration window, and stress-testing the jump-detection threshold across body types and clothing styles.

Finally, I extend my heartfelt appreciation to my family and friends for their unwavering support, patience, and encouragement during the challenging phases of this research. Their belief in my abilities has been a constant source of motivation, and their understanding during the long hours of work has made this achievement possible.

ABSTRACT

Athletic training environments at the school, university, and recreational levels rarely have access to objective, frame-level measurement of jump performance and landing mechanics. Instead, athletes depend on coach observation that is subjective, episodic, and constrained in capacity, while specialised motion-capture laboratories that can deliver such measurements remain prohibitively expensive and physically inaccessible. This component of the NeuroMotion system addresses that gap by delivering an end-to-end video pipeline, skeletal data processing layer, and biomechanical jump analysis engine that runs on a standard webcam-equipped consumer device. The input layer supports both real-time live camera capture using OpenCV and pre-recorded video file upload, with each frame routed sequentially into MediaPipe's BlazePose pose estimation model, which infers thirty-three three-dimensional body landmarks per frame at sub-100ms latency. A temporal smoothing stage applies an exponential moving average with coefficient α equal to 0.7 to suppress single-frame jitter, after which a one-hundred-frame calibration window establishes the athlete's neutral standing position and computes a dynamic jump threshold parameterised across three sensitivity modes corresponding to ankle, knee, and full-torso displacement. Jump events are segmented by detecting threshold crossings in the smoothed trigger-point Y-coordinate, and each detected repetition is evaluated against five biomechanical fault conditions: bilateral knee valgus beyond fifteen degrees, anterior trunk lean exceeding thirty degrees, and bilateral knee-over-toe displacement beyond a normalised offset of one tenth of a frame width. Joint angles are computed using a three-point cosine-rule formulation, and each repetition is awarded a quality score on a zero-to-ten scale with two-point deductions per fault. Cleaned repetition records, including frame indices, scores, fault classifications, and landmark snapshots, are persisted to a local SQLite database and consumed by downstream recommendation and forecasting modules. A data-cleaning layer guards against zero-count sessions through safe division, removes non-finite values through np.isfinite filtering, and clamps engineered features to valid ranges before a linear regression model produces longitudinal forecasts of normalised speed, training volume, and form quality. Evaluation across forty-five test sessions and over six hundred individual jumps shows ninety-three point two percent jump-detection accuracy, ninety-four point one percent agreement with expert posture annotations, sub-100ms median per-frame latency on CPU-only consumer hardware, and a regression coefficient of determination of zero point eight

one for next-session form-quality forecasting. The pipeline therefore demonstrates that accessible, joint-level, longitudinally-aware jump analysis is achievable using only a webcam, an open-source pose model, and a local relational store.

Keywords: *video pipeline, MediaPipe BlazePose, exponential moving average smoothing, jump detection, biomechanical analysis, SQLite persistence, linear regression forecasting*

TABLE OF CONTENTS

Declaration of Candidate and Supervisor	i
Dedication	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
List of Appendices	ix
CHAPTER 1: INTRODUCTION	1
1.1 Background and Literature Survey	1
1.2 Research Gap	6
1.3 Research Problem	9
1.4 Research Objectives	11
CHAPTER 2: LITERATURE REVIEW	13
2.1 Video Ingestion Pipelines for Athletic Analysis	13
2.2 Pose Estimation Models for Single-Camera Capture	16
2.3 Temporal Smoothing of Skeletal Signals	20
2.4 Jump Detection and Repetition Segmentation	23
2.5 Biomechanical Fault Analysis in Plyometric Movements	26
2.6 Performance Forecasting with Regression Models	29
2.7 Summary	31
CHAPTER 3: METHODOLOGY	33
3.1 System Architecture Overview	33
3.2 Video Ingestion Layer	36
3.3 Pose Estimation with MediaPipe BlazePose	39
3.4 Temporal Smoothing via Exponential Moving Average	43

3.5 Calibration Phase and Baseline Establishment 46
3.6 Jump Detection and Sensitivity Modes 49
3.7 Joint Angle Computation Using the Cosine Rule 53
3.8 Biomechanical Fault Detection 56
3.9 Quality Scoring Mechanism 59
3.10 Data Persistence with SQLite 61
3.11 Data Cleaning and Numerical Integrity 64
3.12 Feature Engineering for Performance Forecasting 67
3.13 Linear Regression for Longitudinal Forecasting 70
3.14 Commercialization Aspects of the Product 73
3.15 Testing and Implementation 75
CHAPTER 4: RESULTS AND DISCUSSION 78
4.1 Pipeline Performance Results 78
4.2 Pose Estimation and Smoothing Effectiveness 82
4.3 Jump Detection Accuracy 85
4.4 Form Quality Score Validation 88
4.5 Performance Forecasting Results 91
4.6 Comparison with Existing Approaches 94
4.7 Research Findings 96
4.8 Discussion 98
4.9 Summary of My Individual Contribution 100
CHAPTER 5: CONCLUSION 102
5.1 Summary of the Research 102
5.2 Achievements and Contributions 104
5.3 Limitations 106
5.4 Future Work and Recommendations 108
Reference List 110
Appendices 113

LIST OF FIGURES

Figure 1.1: Conceptual map of the video-pipeline gap addressed by this research 5

Figure 2.1: Taxonomy of single-camera pose estimation approaches reviewed in the literature17

Figure 3.1: NeuroMotion video-pipeline and jump-analysis architecture overview 34

Figure 3.2: Frame ingestion flow for live capture and pre-recorded video modes 37

Figure 3.3: BlazePose 33-landmark skeletal model with the indices used by this component highlighted 40

Figure 3.4: Effect of EMA smoothing on a representative ankle Y-coordinate signal 44

Figure 3.5: One-hundred-frame calibration window and dynamic jump-threshold derivation47

Figure 3.6: Threshold-crossing logic and start/end-frame segmentation for a single jump 51

Figure 3.7: Three-point cosine-rule geometry for joint angle computation 54

Figure 3.8: Five biomechanical fault conditions evaluated during each detected jump 57

Figure 3.9: SQLite schema and end-to-end data flow from frame to recommendation module62

Figure 3.10: Linear regression feature pipeline from raw repetitions to forecast output 71

Figure 4.1: Per-frame latency distribution across the test workload 80

Figure 4.2: Jump-detection accuracy by sensitivity mode and exercise variant 86

Figure 4.3: Predicted versus observed form-quality scores for the linear regression model 92

LIST OF TABLES

Table 1.1: Comparison of existing video-pipeline solutions for athletic analysis 6

Table 2.1: Comparative summary of single-camera pose estimation models 19

Table 2.2: Survey of temporal smoothing strategies in skeletal tracking 22

Table 3.1: Landmark indices consumed by the jump-analysis pipeline 41

Table 3.2: Sensitivity-mode configuration for jump detection 50

Table 3.3: Biomechanical fault thresholds and associated point deductions 58

Table 3.4: SQLite schema for the repetitions table 63

Table 3.5: Engineered features feeding the regression model 68

Table 3.6: Technology stack used in the video-pipeline component 74

Table 4.1: Overall pipeline performance evaluation results 79

Table 4.2: Jump-detection accuracy by sensitivity mode 87

Table 4.3: Linear regression evaluation metrics on the held-out forecast set 93

Table 4.4: Comparison with existing video-pipeline approaches 95

LIST OF ABBREVIATIONS

Abbreviation	Expanded Form
AI	Artificial Intelligence
API	Application Programming Interface
BPM	Beats Per Minute
CNN	Convolutional Neural Network
CPU	Central Processing Unit
EMA	Exponential Moving Average
FPS	Frames Per Second
GPU	Graphics Processing Unit
IoU	Intersection over Union
MAE	Mean Absolute Error
MediaPipe	Google's Cross-Platform Machine Learning Framework
MSE	Mean Squared Error
NaN	Not a Number
OpenCV	Open Source Computer Vision Library
RMSE	Root Mean Squared Error
SQL	Structured Query Language
SQLite	Embedded Relational Database Engine
TTS	Text-to-Speech
YOLO	You Only Look Once

LIST OF APPENDICES

Appendix A BlazePose Landmark Index Reference Used by the Pipeline 113

Appendix B SQLite Schema Specification and Sample Inserts 114

Appendix C Linear Regression Feature-Set Documentation 115

Appendix D Pseudocode for the Jump Detection State Machine 116

CHAPTER 1: INTRODUCTION

1.1 Background and Literature Survey

The measurement of athletic performance has historically been bound to two extremes. At one end, professional motion-capture laboratories employ marker-based optical systems, force plates, and synchronised high-speed camera arrays to derive joint-level biomechanical insights with sub-millimetre precision. At the other end, the overwhelming majority of athletes, from school-level players to recreational lifters, depend on a coach standing on the side of the court or gym floor and forming a subjective impression of how a movement was executed. Between these two extremes there is a striking absence of accessible tooling, particularly for movements such as jumps where the most diagnostically important phase, namely the take-off and landing window, occurs faster than the human eye can reliably evaluate.

Jumping movements are central to a wide range of sports and conditioning programmes, including basketball, volleyball, plyometric training, athletics, and rehabilitation following lower-limb injury. The biomechanical character of a jump is determined by a small set of measurable quantities: the displacement of the centre of mass, the alignment of the knees relative to the toes during landing, the inward collapse of either knee, and the degree of forward trunk lean during take-off. Each of these quantities can in principle be derived from a single camera if landmarks corresponding to the ankles, knees, hips, and shoulders can be located in every frame and tracked through time. The technical question then becomes how reliably and how cheaply that landmark localisation can be performed, and whether the resulting signal is clean enough to drive automated repetition counting and form scoring.

The release of MediaPipe BlazePose by Bazarevsky et al. in 2020 marked a significant turning point for accessible single-camera pose estimation [1]. The model produces thirty-three three-dimensional body landmarks at sub-100ms latency on consumer CPUs, exposing not only the seventeen keypoints inherited from the COCO format but also additional anatomical reference points at the hands, feet, and face. For jump analysis, the inclusion of dedicated foot landmarks and explicit hip and shoulder midline points is particularly relevant, since jump quality is most cleanly characterised by relative motion between these reference frames. The fact that BlazePose

runs on commodity hardware, without requiring a discrete GPU or specialised hardware, makes it a compelling foundation for a pipeline targeted at school-level and recreational athletes.

However, raw pose estimation output alone is not equivalent to a usable analysis pipeline. Several engineering layers stand between a model that emits landmarks and a system that can tell an athlete that they completed twenty-three jumps with an average form score of seven point four, that two of those jumps exhibited right knee valgus, and that their next session is forecast to show a slight improvement in form quality if their current training cadence is maintained. Each frame's landmark prediction contains a meaningful amount of single-frame jitter, arising from motion blur during fast take-offs, partial occlusion when the athlete's arms cross in front of the torso, and inherent uncertainty in the detector's heatmap regression. Without temporal smoothing, that jitter directly contaminates any downstream threshold-crossing logic used to identify jump events. Without a calibration phase that establishes what neutral standing looks like for the specific athlete in front of the camera, threshold values cannot be set in a body-agnostic way. Without robust persistence and data cleaning, the records that emerge from an exercise session will not be in a state fit for feature engineering, much less for regression-based forecasting.

Existing literature on video-based athletic analysis tends to address one or two of these layers in isolation rather than the pipeline as a whole. Liao et al. presented a deep-learning framework for assessing physical rehabilitation exercises that integrated pose estimation with a deep network trained to score movement quality, but their work did not include a full ingestion-to-forecasting pipeline and required a curated dataset for each new exercise [2]. Cao et al. introduced OpenPose, demonstrating real-time multi-person 2D pose estimation, but the system was designed primarily as a research tool rather than as a building block for a deployed analysis stack [3]. Commercial fitness applications often include rep counters but treat the underlying signal as a black box and provide no biomechanical interpretation, no persistence of structured records, and no longitudinal analysis.

The present work focuses on closing that engineering gap. Rather than introducing a new pose estimation model or competing with optical motion-capture laboratories on raw accuracy, this component of the NeuroMotion system contributes a coherent, reproducible video-ingestion-to-forecasting pipeline. Each layer in that pipeline has been designed to remain accurate and interpretable on consumer hardware while producing structured records that can be consumed by

both human users and downstream machine-learning modules contributed by other members of the team. The design decisions, particularly the choice of α equal to 0.7 for exponential moving average smoothing, the use of a one-hundred-frame calibration window, and the three-mode sensitivity scheme for jump-threshold computation, are the result of empirical tuning across the volunteer dataset described in Chapter 3.

Figure 1.1 visualises the pipeline-level gap addressed in this dissertation. On the left, the input modalities consist of either a live webcam stream or a pre-recorded video file. In the middle, three traditional categories of system are shown: motion-capture laboratories, which provide accurate analysis but are inaccessible; commercial fitness applications, which are accessible but offer no biomechanical interpretation; and pure pose estimation libraries, which provide the necessary primitives but do not deliver an end-to-end pipeline. On the right, the output that an athlete actually needs is shown: a per-repetition record with a quality score, fault classifications, and a forecast for the next session. The dissertation argues that the missing element in current practice is the integrated middle layer that turns raw landmark streams into that structured output.

Figure 1.1: Conceptual map of the video-pipeline gap addressed by this research. Source: Author.

This research is grounded in the observation that the integration work itself, comprising signal smoothing, calibration, threshold-based segmentation, fault detection, scoring, persistence, cleaning, feature engineering, and regression-based forecasting, constitutes a substantive engineering and research contribution. Each component is technically straightforward in isolation, but their composition into a coherent pipeline that holds up under varied lighting, body types, and session durations is non-trivial and is reported in detail in subsequent chapters.

Table 1.1 contrasts the four main categories of system on the four most diagnostically important attributes for the present research.

System Category	Joint-Level Analysis	Single Camera	Real-Time	Persistence and Forecasting
Motion-capture laboratory	Yes	No	No	No
Commercial fitness apps	No	Yes	Yes	Limited

Standalone pose libraries (BlazePose, OpenPose)	Partial	Yes	Yes	No
NeuroMotion video-pipeline component	Yes	Yes	Yes	Yes

Table 1.1: Comparison of existing video-pipeline solutions for athletic analysis.

1.2 Research Gap

A close reading of the existing literature and commercial landscape reveals four overlapping but distinct gaps that this component of the NeuroMotion system is designed to address. The first gap is the absence of an integrated, calibration-aware, threshold-based jump segmentation pipeline that operates on a single webcam stream and produces per-repetition records suitable for downstream consumption. While individual building blocks exist, including pose estimation libraries, smoothing filters, and event-detection algorithms, no widely available open-source pipeline composes these pieces into a single deployable artefact specifically tuned for jump biomechanics.

The second gap concerns the body-agnostic establishment of jump thresholds. Athletes vary in height, limb proportions, and standing posture. A fixed pixel offset that correctly identifies a jump for one athlete will misclassify shallow movements for another or miss legitimate jumps for a third. Existing public implementations either assume a fixed threshold or require manual configuration each time, both of which are unacceptable in a deployable product. The one-hundred-frame calibration window introduced in Section 3.5 of this dissertation addresses this gap directly by deriving a per-athlete reference position from which the jump threshold is offset in a body-aware way.

The third gap concerns the persistence and longitudinal tracking of jump records. The vast majority of pose-based analysis work treats each session as an isolated computation, with no structured record left behind after the session ends. For an athlete who trains over weeks or months, this is a significant limitation: there is no foundation upon which forecasting, fatigue analysis, or trend detection can be built. The SQLite-backed persistence layer described in Section 3.10 produces a structured, queryable record of every repetition, including its frame range, point score, fault classifications, and associated landmark snapshots, providing exactly that foundation.

The fourth gap is the absence of accessible longitudinal forecasting. While elite training environments may have access to sport-science staff who can interpret repeated test results, recreational and school-level athletes typically do not. A simple linear regression model trained on engineered features such as normalised speed, volume, and form quality offers a low-overhead route to producing interpretable session-over-session forecasts. The model does not need to compete with deep ensembles to be useful; it needs to be reliable, interpretable, and trained on clean data, which is exactly what the data-cleaning layer in Section 3.11 ensures.

These four gaps motivate the four research objectives stated in Section 1.4. Together, they define a contribution that is engineering-rich rather than novelty-rich: the dissertation does not claim to introduce a new pose estimation model, but it does claim to introduce, validate, and document a coherent pipeline that fills a measurable accessibility gap in current practice.

1.3 Research Problem

The research problem addressed by this component of the NeuroMotion system can be stated succinctly. Given only a standard webcam and a consumer-grade computing device, how can an end-to-end pipeline be designed and implemented such that the system reliably ingests video frames, extracts stable skeletal landmarks, segments jump events from a continuous stream, evaluates each event against biomechanical fault conditions, persists the resulting records, and produces a longitudinal forecast of future performance, all while preserving sub-100ms per-frame latency and exposing well-formed structured records to downstream modules?

This problem is non-trivial for several reasons. The pose estimation model itself produces noisy single-frame predictions that must be temporally smoothed without introducing perceptible lag. The threshold used to identify a jump must be derived from a calibration phase that is brief enough to remain user-friendly yet long enough to produce a stable baseline. The fault-detection logic must handle five distinct biomechanical conditions, each requiring its own three-point cosine-rule angle computation, while remaining robust to occasional landmark dropouts. The persistence layer must guard against numerical pathologies such as zero-count sessions and non-finite values that would corrupt downstream feature engineering. And the forecasting model must be trained on data that has been thoroughly cleaned and clamped to valid ranges, lest the regression coefficients be dragged towards meaningless directions by outliers.

The problem is also non-trivial from an integration perspective. The pipeline does not exist in isolation; it produces inputs for the recommendation module, the voice-feedback module, and the posture-correction module developed by other members of the team. The schema of the persisted records, the timing of when records become available, and the granularity of the fault classifications must all be designed in cooperation with those downstream modules. A pipeline that produced perfect results in isolation but did not align with the consumption patterns of the rest of the system would not satisfy the project's requirements.

Finally, the problem must be solved within practical resource constraints. The system targets consumer hardware without GPU acceleration, runs in environments with potentially poor lighting, and is intended to operate over sessions of fifteen to forty-five minutes without crashing or developing memory leaks. These operational constraints rule out approaches that, for example, require offline batch processing of an entire session before producing any output, or that maintain large in-memory buffers of every frame seen so far.

1.4 Research Objectives

In light of the research problem stated above, the present component of the NeuroMotion system pursues five specific, measurable objectives, each of which is revisited and evaluated in Chapter 4.

The first objective is to design and implement a dual-mode video ingestion layer that supports both real-time live camera capture and pre-recorded video file upload through a unified interface, with a measurable per-frame ingestion latency below twenty milliseconds. This objective is addressed by the OpenCV-based ingestion module described in Section 3.2 and evaluated in Section 4.1.

The second objective is to integrate the MediaPipe BlazePose pose estimation model into the pipeline in a manner that delivers stable thirty-three-landmark predictions at sub-100ms end-to-end latency on consumer CPUs, including the smoothing and downstream evaluation stages. This objective is addressed by the integration described in Section 3.3 and evaluated in Sections 4.1 and 4.2.

The third objective is to derive and validate a calibration-aware, body-agnostic jump-detection algorithm with three configurable sensitivity modes corresponding to ankle, knee, and full-torso

displacement, and to demonstrate that the algorithm correctly segments at least ninety percent of jumps performed by athletes of varying body types in the test set. This objective is addressed by the calibration and detection logic in Sections 3.5 and 3.6 and evaluated in Section 4.3.

The fourth objective is to implement a five-condition biomechanical fault-detection layer based on three-point cosine-rule joint angle computations, accompanied by a deterministic zero-to-ten quality scoring mechanism, and to demonstrate that the resulting per-repetition scores agree with expert annotations at a frame-level accuracy of at least ninety percent. This objective is addressed in Sections 3.7 through 3.9 and evaluated in Section 4.4.

The fifth objective is to design a SQLite-backed persistence layer with explicit data-cleaning safeguards and a feature-engineering stage that feeds a linear regression model for longitudinal performance forecasting, with the requirement that the regression model achieve a coefficient of determination of at least zero point seventy on a held-out forecast set. This objective is addressed in Sections 3.10 through 3.13 and evaluated in Sections 4.5 and 4.6.

CHAPTER 2: LITERATURE REVIEW

2.1 Video Ingestion Pipelines for Athletic Analysis

Video ingestion in the context of athletic analysis has historically been treated either as a trivial pre-processing step or, on the opposite extreme, as a hardware engineering problem solvable only by professional motion-capture installations. Between these poles, the practical question of how to reliably stream frames from either a live webcam or a pre-recorded file into an analysis pipeline has received less explicit attention than the more visible problems of pose estimation and downstream classification. The few works that engage with this layer in a serious way emphasise three recurring concerns: timing consistency across heterogeneous sources, frame-rate normalisation, and graceful failure when capture devices misbehave.

Bradski's foundational work on OpenCV established the canonical interface for video ingestion in research and industrial applications, exposing the cv2.VideoCapture abstraction that allows the same downstream code to operate on a webcam stream, a video file, or a sequence of images [4]. The simplicity of this interface masks an important architectural advantage: pipelines built against it are inherently mode-agnostic, allowing the same analysis logic to operate on live and recorded inputs without modification. The present work adopts this abstraction directly and extends it with a thin orchestration layer that detects the source type, normalises frame timing, and emits a uniform frame stream into the pose estimation stage.

More recent work in sports analytics has emphasised the need for ingestion layers that are robust to the realities of consumer hardware. Bridgeman et al. reported on multi-person 3D pose estimation in sports environments and noted that frame drops during high-motion sequences are a frequent source of analysis errors that are easily mistaken for genuine biomechanical events [5]. Their recommendation, which the present pipeline follows, is to record actual frame indices and capture timestamps rather than assuming a constant frame rate, since the latter assumption silently corrupts any rate-based metric such as repetitions per minute.

In commercial contexts, fitness applications have largely converged on a similar dual-mode design: a live capture mode for real-time feedback and an upload mode for post-session analysis. The architectural pattern adopted by the present component aligns with this convergent design but

extends it with explicit calibration handling that distinguishes between the two modes. In the live mode, calibration is interactive and consumes the first one hundred frames of the session; in the upload mode, calibration is performed on the first one hundred frames of the supplied video, with appropriate user-facing messaging if the supplied video does not contain a sufficiently long neutral standing window.

2.2 Pose Estimation Models for Single-Camera Capture

The literature on single-camera pose estimation has matured significantly over the past decade, with a clear progression from heatmap-based two-stage architectures to lighter, real-time-oriented designs intended for deployment on consumer hardware. Cao et al. introduced OpenPose, which used part affinity fields to associate body parts across multi-person scenes and remains influential as a research baseline [3]. While OpenPose is highly accurate, its computational footprint is poorly matched to single-user fitness applications running on a laptop, where multi-person association is unnecessary overhead and real-time latency is paramount.

Bazarevsky et al. introduced BlazePose specifically to address the deployment gap between research-grade pose estimation and on-device inference [1]. The model uses a two-stage tracker-detector architecture in which a relatively lightweight detector locates the person in the frame, after which a tracker refines thirty-three landmarks frame-to-frame using the previous frame's prediction as a starting point. This design produces sub-100ms per-frame inference on consumer CPUs and exposes a richer landmark set than the COCO-standard seventeen-keypoint format. For the present pipeline, the inclusion of dedicated foot landmarks at indices 27 and 28 (left and right ankle) and the explicit shoulder and hip landmarks at indices 11, 12, 23, and 24 are particularly valuable, since jump biomechanics is most naturally expressed in terms of the relative motion of these reference points.

Fang et al. presented RMPE, a regional multi-person pose estimation framework that emphasised robustness to detection errors through symmetric pose nonmaximum suppression [6]. While the multi-person focus is not directly relevant to the single-athlete jump-analysis use case, the underlying observation that pose models are highly sensitive to upstream detection quality is directly applicable. The present pipeline mitigates this sensitivity by relying on the BlazePose

tracker's frame-to-frame stability and by applying exponential moving average smoothing as a second line of defence against transient prediction errors.

More recent work has explored YOLO-family architectures with native pose-estimation heads. Wang et al. introduced YOLOv7, demonstrating that single-stage object detectors can be extended to pose estimation tasks while maintaining their characteristic real-time performance [7]. Jocher and colleagues subsequently released YOLOv8, which natively supports both classification and keypoint estimation in a unified training pipeline [8]. While the NeuroMotion system as a whole employs YOLOv8 in its custom-trained exercise-classification and keypoint-estimation models for squats and push-ups, the jump-analysis component reported in this dissertation deliberately uses BlazePose for its lower deployment overhead and for its richer thirty-three-landmark output, which is well matched to the explicit ankle, knee, hip, and shoulder references required by the jump fault-detection logic.

Table 2.1 summarises the four most commonly cited single-camera pose models, their measured CPU latency on a representative consumer laptop, the number of landmarks they expose, their characteristic strengths, and their associated trade-offs.

Model	**Latency (CPU)**	**Landmarks**	**Strength**	**Trade-off**
OpenPose	~250ms	18 (COCO)	Multi-person robustness	Heavy compute footprint
BlazePose	~70ms	33 (extended)	Real-time, rich landmark set	Single-person only
YOLOv8-Pose	~90ms	17 (COCO)	Unified detection and pose	Fewer landmarks than BlazePose
RMPE / AlphaPose	~180ms	17 (COCO)	Symmetric NMS robustness	Two-stage latency penalty

Table 2.1: Comparative summary of single-camera pose estimation models.

2.3 Temporal Smoothing of Skeletal Signals

Single-frame pose predictions, even from state-of-the-art models, exhibit positional jitter on the order of several pixels per frame. For static or slowly moving keypoints, this jitter is benign and largely invisible to the user. For threshold-based event-detection logic, however, even small amounts of high-frequency noise can produce spurious threshold crossings that are misinterpreted as jump events. The literature on temporal smoothing of skeletal signals offers several candidate filters, each with distinct latency, computational, and signal-fidelity trade-offs.

The simplest smoothing approach is the moving average filter, which replaces each landmark's position with the unweighted mean of its recent values across a fixed window. Moving average filters are computationally cheap and easy to implement but introduce a fixed lag equal to roughly half the window length, which is unacceptable for real-time correction systems where users expect near-instantaneous feedback. Median filtering, while more robust to single-frame outliers, exhibits the same lag characteristic and is more expensive to compute.

Exponential moving average (EMA) filtering, introduced into the broader signal-processing literature decades ago and adopted in pose-tracking contexts by several authors, offers an attractive balance for real-time skeletal smoothing. The EMA filter assigns exponentially decreasing weights to past observations through a single coefficient α, where α equal to one corresponds to no smoothing and α approaching zero corresponds to extreme smoothing. The recursive nature of the filter means that only the previous smoothed value need be retained, making the memory footprint negligible. Liao et al. used EMA-style smoothing in their rehabilitation exercise scoring framework with a coefficient tuned for clinical applications [2], and the present pipeline applies the same filtering family with α equal to 0.7, a value selected through empirical evaluation reported in Chapter 4.

More sophisticated alternatives include the Kalman filter, which models the underlying signal as a linear dynamical system with Gaussian noise. Kalman filtering is the principled choice when an explicit motion model is available, and it has been used effectively in inertial-sensor-based motion analysis. For pose-based jump analysis, however, the motion model required to make Kalman filtering productive (essentially a per-landmark velocity and acceleration model) introduces significant complexity for a marginal accuracy gain, particularly because the underlying BlazePose tracker already incorporates frame-to-frame consistency. The present pipeline therefore prefers the simpler EMA approach.

Table 2.2 summarises the most commonly considered smoothing filters along the four axes most relevant to pose-based jump analysis.

Filter Family	Lag	Compute Cost	Memory	Suitability for Jump Analysis
Moving average (window 5)	~2 frames	Low	5 values per landmark	Poor (visible lag)
Median (window 5)	~2 frames	Medium	5 values per landmark	Poor (lag plus expense)
EMA ($\alpha = 0.7$)	Sub-frame	Very low	1 value per landmark	Excellent
Kalman (linear)	Sub-frame	Medium	Full state per landmark	Good but heavy
One-Euro filter	Sub-frame	Low	2 values per landmark	Good alternative

Table 2.2: Survey of temporal smoothing strategies in skeletal tracking.

2.4 Jump Detection and Repetition Segmentation

Identifying individual jump events within a continuous stream of frames is, in essence, a one-dimensional event-segmentation problem operating on the smoothed Y-coordinate of a chosen trigger point. The literature on this problem in the pose-based fitness context is sparse and largely confined to commercial implementations whose details are not publicly available. The closest analogues come from the gait-analysis and inertial-sensor-based activity-recognition literature, where step detection from accelerometer signals shares the same fundamental structure as jump detection from pose-based vertical displacement signals.

Ignatov reported on real-time human activity recognition from accelerometer data using convolutional neural networks, demonstrating that learned classifiers can achieve high accuracy on activity-detection tasks but at a substantial computational cost [9]. For the simpler case of detecting jump events from a smoothed positional signal, deep learning approaches offer a poor

trade-off: a threshold-crossing rule operating on the EMA-smoothed trigger-point Y-coordinate is interpretable, near-zero-cost, and readily auditable. The present pipeline therefore adopts the threshold-crossing approach and devotes its complexity budget elsewhere, principally to the calibration logic that ensures the threshold is set appropriately for each athlete.

A subtle but important point in jump segmentation is the choice of trigger point. The naive choice is the centre of mass, but this point cannot be observed directly and must be approximated, typically by the mean of the hip and shoulder midpoints. The slightly better choice for jump detection specifically is one of the lower-body landmarks, since these exhibit the largest absolute vertical displacement during a jump and therefore the highest signal-to-noise ratio. The present pipeline allows the operator to select between three trigger-point modes: ankle midpoint (low sensitivity, designed for shallow jumps), knee midpoint (medium sensitivity, the default), and full-torso displacement (high sensitivity, suitable for full-body plyometric work). This three-mode design is, to the author's knowledge, novel in the publicly documented literature on pose-based jump segmentation.

2.5 Biomechanical Fault Analysis in Plyometric Movements

The biomechanical literature on plyometric and jumping movements is extensive and has identified a stable set of fault conditions whose presence is associated with increased injury risk and reduced performance. Knee valgus, which is the inward collapse of the knee during landing, has been linked in numerous prospective studies to elevated risk of anterior cruciate ligament injury, particularly in female athletes [10]. The conventional clinical threshold for considering knee valgus to be biomechanically significant is a deviation of approximately fifteen degrees from the ideal hip-knee-ankle alignment, which is the threshold adopted in the present fault-detection logic.

Anterior trunk lean, defined as the forward inclination of the torso relative to the vertical, is similarly well-characterised in the literature as a marker of hip-extensor weakness or compensatory loading patterns. Excessive trunk lean during a jump take-off shifts load away from the gluteal complex and onto the lower lumbar spine, increasing both injury risk and energetic inefficiency. The threshold adopted in the present pipeline, namely thirty degrees of forward lean as measured

by the shoulder-hip-knee angle, is consistent with the values reported in clinical sports-medicine guidelines.

Knee-over-toe displacement, sometimes referred to as anterior knee translation, is the third condition evaluated by the fault-detection logic. The condition is detected when the knee landmark's normalised X-coordinate exceeds the corresponding ankle landmark's X-coordinate by more than one tenth of a frame width. This threshold is anatomically derived: a normal landing has the knee tracking approximately above the toes, with small forward translation absorbed by ankle dorsiflexion. Translations beyond approximately a tenth of a frame width consistently correspond to athletes whose centre of mass has tipped forward beyond the limits of safe landing mechanics.

Each of the five fault conditions evaluated by the pipeline (left knee valgus, right knee valgus, anterior trunk lean, left knee-over-toe, and right knee-over-toe) thus has an established biomechanical basis. The contribution of the present work is not the identification of these conditions but the implementation of a real-time, single-camera system that evaluates all five within each detected jump and produces an interpretable point score reflecting their combined effect.

2.6 Performance Forecasting with Regression Models

Forecasting future athletic performance from a record of past sessions is a regression problem whose simplest and most interpretable form is ordinary least squares applied to a small number of engineered features. While the broader machine-learning literature has explored more sophisticated approaches including gradient-boosted trees, recurrent neural networks, and transformer-based time-series models, these alternatives all require substantially more training data than is available from a single athlete's first few sessions and offer little interpretability advantage in exchange. The present pipeline therefore adopts linear regression as the forecasting model, deliberately trading predictive ceiling for interpretability and data-efficiency.

The features fed into the regression model, described in detail in Section 3.12, are normalised speed (repetitions per minute relative to a fifty-rep baseline), training volume (total repetitions per session), and form quality score (mean per-repetition score across the session). Each feature is

clamped to a documented valid range and verified to be finite before being passed to the model. The regression coefficients themselves are then exposed to downstream modules and to the user, allowing the interpretive question of which feature is most strongly predictive of future form quality to be answered directly from the trained model.

2.7 Summary

The literature surveyed in this chapter establishes that the individual building blocks required for the present pipeline are well understood. Single-camera pose estimation has matured to the point where models such as BlazePose run reliably on consumer CPUs. Temporal smoothing through exponential moving average filters has a long track record in real-time signal processing and offers the best latency-fidelity trade-off for the present use case. Threshold-based jump segmentation, while less extensively studied than its inertial-sensor analogue, is a well-understood technique that benefits from the explicit calibration logic introduced in this work. Biomechanical fault thresholds for knee valgus, anterior trunk lean, and knee-over-toe displacement are well established in the sports-medicine literature, and linear regression provides a defensible, interpretable forecasting model for the small-data regime that characterises individual-athlete training records.

What is missing from the existing literature is the integration of these building blocks into a coherent, deployable pipeline tuned for jump analysis specifically and accessible to users without specialised equipment. The next chapter describes how this integration was performed, how each layer was tuned through empirical evaluation on the volunteer dataset, and how the final pipeline was designed to expose well-formed structured records to the downstream recommendation and forecasting modules.

CHAPTER 3: METHODOLOGY

3.1 System Architecture Overview

The video-pipeline component of NeuroMotion is implemented as a modular Python application that integrates with the broader Streamlit-based front end developed by the team. Internally, the component is organised into nine sequential stages that together transform a raw video frame into a persisted, scored, and forecastable repetition record. These stages are: ingestion, pose estimation, temporal smoothing, calibration, jump detection, joint angle computation, fault detection, scoring, and persistence. A tenth stage, feature engineering and forecasting, operates over the persisted records to produce session-over-session predictions of athletic performance.

The architectural separation of these stages reflects two design principles. The first is fail-isolation: a fault in one stage, such as a transient pose-estimation dropout, should not cascade into invalid persisted records. Each stage exposes a documented input and output contract, and the boundaries between stages enforce the type and range invariants that downstream stages depend on. The second principle is observability: each stage emits structured logging output that is sufficient to reconstruct what happened during a session even after the fact, which proved essential during the development and tuning phases.

Figure 3.1 illustrates the complete architecture. On the left, two input modes feed into a unified ingestion abstraction that emits a frame stream into the pose estimation engine. The stream then passes through smoothing, calibration, and jump detection in sequence. For each detected jump, the pipeline branches into the joint-angle, fault-detection, and scoring stages, after which the resulting record is persisted to SQLite. The cleaning, feature-engineering, and regression layers operate on the persisted records and produce forecasts that are exposed to the user interface and to the recommendation module developed by another team member.

Figure 3.1: NeuroMotion video-pipeline and jump-analysis architecture overview. Source: Author.

Backend processing is performed entirely on the local device. This decision eliminates network latency, removes a class of failure modes associated with intermittent connectivity, and preserves user privacy by avoiding the transmission of video data to external services. Streamlit provides the

web interface layer, rendering live results and historical dashboards in the browser, while the Python backend handles all computational work.

3.2 Video Ingestion Layer

The ingestion layer is implemented as a thin orchestration wrapper around OpenCV's cv2.VideoCapture class. The wrapper exposes a single iterator interface that yields successive frames regardless of whether the underlying source is a webcam (referred to in the OpenCV API by the integer index 0) or a video file path supplied by the user. Internally, the wrapper detects the source type at construction time and configures the underlying capture object appropriately. For the webcam mode, the capture object is configured to a target frame rate of thirty frames per second and a resolution of seven hundred and twenty pixels by default, with fallback to the webcam's native resolution if the requested resolution is unsupported. For the file-upload mode, the capture object respects the embedded frame rate and resolution of the source video.

Each frame yielded by the ingestion layer is accompanied by two metadata fields: the integer frame index since session start and the high-resolution timestamp at which the frame was captured. These fields are critical for downstream rate-based metrics such as repetitions per minute, since the inferred frame rate from a webcam can drift under load and a constant-rate assumption would silently corrupt the resulting metric. The ingestion layer also emits an explicit end-of-stream sentinel when the underlying source is exhausted, allowing downstream stages to flush any in-flight state and produce a clean final record.

Figure 3.2 visualises the ingestion flow. The two source paths converge at the wrapper boundary, after which the downstream stages see a uniform frame stream regardless of source. This uniformity is the architectural foundation that allows the same analysis logic to operate on both live and recorded inputs without modification.

Figure 3.2: Frame ingestion flow for live capture and pre-recorded video modes. Source: Author.

The ingestion layer also handles the practical reality that webcam capture occasionally fails to produce a frame within the expected timing window. When this happens, the wrapper retries the capture call up to three times with exponential backoff before raising an exception, and the retry

events are logged for later analysis. During the development phase, this retry logic was found to convert what would otherwise be visible session interruptions into invisible, recoverable jitter.

3.3 Pose Estimation with MediaPipe BlazePose

The pose estimation stage receives each frame from the ingestion layer and forwards it to the MediaPipe BlazePose model in its full body configuration with the model_complexity parameter set to one. This configuration corresponds to the medium-quality preset, which strikes a balance between inference latency and landmark accuracy. The model returns a list of thirty-three landmarks, each with normalised x, y, and z coordinates in the range zero to one (where the X and Y coordinates are normalised to the frame width and height respectively, and the Z coordinate is normalised to the frame width as a relative depth estimate) along with a visibility score also in the range zero to one.

Of the thirty-three landmarks emitted by BlazePose, the present component consumes exactly eight: the left and right ankles at indices 27 and 28, the left and right knees at indices 25 and 26, the left and right hips at indices 23 and 24, and the left and right shoulders at indices 11 and 12. These eight landmarks are sufficient to compute every joint angle and threshold required by the jump-detection and fault-detection logic. Landmarks outside this set are still received and stored in the per-frame snapshot for forensic purposes but are not used in the active analysis path. Figure 3.3 highlights the eight consumed landmarks against the full BlazePose skeleton.

Figure 3.3: BlazePose 33-landmark skeletal model with the indices used by this component highlighted. Source: Author.

Table 3.1 summarises the eight consumed landmarks, their BlazePose indices, and their specific roles within the pipeline.

Landmark	Index	Role in Pipeline
Left shoulder	11	Trunk lean angle (upper reference)
Right shoulder	12	Trunk lean angle (upper reference)
Left hip	23	Trunk lean angle (vertex), valgus reference

Right hip	24	Trunk lean angle (vertex), valgus reference
Left knee	25	Knee valgus, knee-over-toe, trigger point
Right knee	26	Knee valgus, knee-over-toe, trigger point
Left ankle	27	Knee valgus, knee-over-toe, low-sensitivity trigger
Right ankle	28	Knee valgus, knee-over-toe, low-sensitivity trigger

Table 3.1: Landmark indices consumed by the jump-analysis pipeline.

Each call to the pose estimation model is timed and recorded for downstream latency analysis. The measured median inference time on the reference consumer laptop used for development is sixty-eight milliseconds per frame, well within the sub-100ms latency budget set by the second research objective. The 95th-percentile inference time is ninety-one milliseconds, which is the worst-case figure considered acceptable for real-time feedback.

In rare cases the BlazePose detector fails to locate the athlete in the frame, typically when the athlete leaves the frame entirely or when severe motion blur during a fast take-off temporarily defeats the detector. In these cases, the model returns a null result, which the pipeline handles by carrying forward the previous smoothed landmark values for one frame, after which the calibration and jump-detection logic suspends until valid landmarks are once again returned. This behaviour avoids spurious jump events being triggered by missing data and is logged for forensic review.

3.4 Temporal Smoothing via Exponential Moving Average

The smoothing stage applies an exponential moving average filter to each of the eight consumed landmarks' x and y coordinates. For a landmark whose raw coordinate at frame t is denoted P(t), the smoothed coordinate P-prime at frame t is computed using the standard EMA recurrence:

$$P'(t) = \alpha \times P(t) + (1 - \alpha) \times P'(t - 1)$$

The smoothing coefficient α is set to 0.7 across all landmarks. This value was selected through empirical evaluation on a development subset of twelve volunteer sessions, in which α values ranging from 0.4 to 0.95 in steps of 0.05 were each evaluated against two complementary criteria: the standard deviation of the smoothed signal during static standing windows (where lower is

better, indicating effective jitter suppression) and the time-to-track of the smoothed signal during simulated step changes (where lower is better, indicating that the filter does not introduce visible lag during real motion).

At α equal to 0.7, the standard deviation during static standing windows fell to approximately twenty-three percent of the unsmoothed value, while the simulated time-to-track remained below one and a half frames. Lower values of α produced better static standard deviation but unacceptable lag, while higher values failed to suppress enough jitter to prevent spurious threshold crossings during the calibration window. The chosen value therefore represents the empirical sweet spot for the present pipeline.

The Z-coordinate is not smoothed in the present pipeline, since the jump-detection and fault-detection logic operates entirely in the X-Y plane. The Z-coordinate is, however, retained in the per-frame snapshot for forensic analysis and for potential future use by depth-aware extensions of the fault-detection logic.

Figure 3.4 illustrates the effect of the smoothing filter on a representative ankle Y-coordinate signal across a twelve-second window containing three jumps. The raw signal exhibits visible single-frame jitter even during the static standing windows between jumps, while the smoothed signal preserves the jump events themselves while suppressing the high-frequency noise that would otherwise contaminate the threshold-crossing logic.

Figure 3.4: Effect of EMA smoothing on a representative ankle Y-coordinate signal. Source: Author.

3.5 Calibration Phase and Baseline Establishment

Before any jump-detection logic can operate, the pipeline must establish a per-athlete reference position from which jump thresholds can be derived. The calibration phase serves this purpose by collecting one hundred frames of smoothed landmark data while the athlete stands in a neutral, upright posture facing the camera. At thirty frames per second, this corresponds to a calibration window of approximately three point three seconds, which is short enough to feel non-intrusive in user testing but long enough to produce a stable mean baseline that is robust to incidental sway.

During the calibration phase, the pipeline accumulates the smoothed Y-coordinates of the trigger-point landmark (which depends on the selected sensitivity mode, as described in Section 3.6) and computes the running mean. At the end of the calibration phase, this running mean is recorded as the calibrated baseline Y-coordinate, denoted Y_baseline. The dynamic jump threshold Y_threshold is then computed as a fixed offset above this baseline:

$$Y_threshold = Y_baseline - \Delta_mode$$

Where Δ_mode is a sensitivity-mode-specific offset whose value is described in Section 3.6 and Table 3.2. The minus sign reflects the fact that, in image space, the Y-axis grows downward, so an upward jump (which is what we want to detect) corresponds to a decrease in the Y-coordinate. A frame whose smoothed trigger-point Y-coordinate falls below Y_threshold is therefore considered to be inside an active jump event.

If the pipeline detects significant motion of the trigger-point during the calibration window, signalled by the standard deviation of the accumulated values exceeding a fixed cap, the calibration is rejected and the athlete is asked to stand still and retry. This guard prevents the system from establishing a baseline that is itself contaminated by motion, which would in turn produce a poorly-calibrated threshold and either spurious or missed jumps. Figure 3.5 visualises the calibration window and the derivation of Y_threshold from Y_baseline and Δ_mode.

Figure 3.5: One-hundred-frame calibration window and dynamic jump-threshold derivation. Source: Author.

3.6 Jump Detection and Sensitivity Modes

With the dynamic threshold established by the calibration phase, jump detection proceeds as a simple state machine over the smoothed trigger-point Y-coordinate. The state machine has two states: idle, in which the trigger-point Y-coordinate is at or above the baseline; and active, in which the trigger-point Y-coordinate is below the threshold. A transition from idle to active is recorded as the start of a jump and the corresponding frame index is stored as start_frame. A transition from active back to idle, gated by the trigger-point Y-coordinate returning to or above the baseline, is recorded as the end of a jump and the corresponding frame index is stored as end_frame.

The pipeline supports three sensitivity modes that determine both the trigger-point landmark and the offset Δ_mode used to compute the threshold. The low-sensitivity mode uses the midpoint of the two ankle landmarks as the trigger point and applies a small offset, making the system trigger only on jumps with substantial vertical foot displacement. The medium-sensitivity mode, which is the default, uses the midpoint of the two knee landmarks and applies an intermediate offset, suitable for typical squat-jump and counter-movement-jump exercises. The high-sensitivity mode uses the midpoint of the four torso landmarks (both shoulders and both hips) and applies a larger offset, making the system trigger on full-body plyometric work where the entire torso displaces upward.

Table 3.2 documents the configuration of each sensitivity mode, including the trigger-point landmark used and the corresponding offset value as a percentage of frame height.

Mode	Trigger Point	Offset Δ	Best Suited For
Low	Ankle midpoint (idx 27, 28)	Small (≈3% of frame height)	Shallow jumps, foot-only movements
Medium	Knee midpoint (idx 25, 26)	Intermediate (≈6% of frame height)	Squat jumps, counter-movement jumps
High	Torso centroid (idx 11,12,23,24)	Large (≈9% of frame height)	Full-body plyometric work

Table 3.2: Sensitivity-mode configuration for jump detection.

Figure 3.6 illustrates the threshold-crossing logic for a single jump. The smoothed trigger-point Y-coordinate descends below Y_threshold at the start of the take-off, remains below throughout the airborne phase, and returns above the baseline at the end of the landing phase. The corresponding start_frame and end_frame indices are recorded for later use by the joint-angle and fault-detection stages.

Figure 3.6: Threshold-crossing logic and start/end-frame segmentation for a single jump. Source: Author.

To prevent the system from registering very short, non-jump motion as a spurious repetition, the pipeline imposes a minimum jump duration of five frames. Active windows shorter than this

minimum are discarded and not persisted. This guard was added after early development testing showed that incidental hand or torso movement during the calibration window could occasionally trigger one-frame or two-frame active windows that polluted the per-session repetition count.

3.7 Joint Angle Computation Using the Cosine Rule

Each detected jump triggers a joint-angle computation pass that evaluates the five biomechanical fault conditions described in Section 3.8. The angles required for these conditions are computed using a three-point cosine-rule formulation that takes three landmark positions A, B, and C, where B is the vertex of the angle, and returns the interior angle at B. The formulation is the standard application of the law of cosines: the angle θ at B satisfies the relation cos θ equals the dot product of the vectors BA and BC divided by the product of their norms. In code, this is computed as:

$$\theta = \arccos((BA \cdot BC) / (||BA|| \times ||BC||))$$

Where BA and BC are the two-dimensional vectors from B to A and from B to C respectively. The arccos function is supplied by the underlying NumPy implementation, which clamps its argument to the valid range [-1, 1] internally to handle the small floating-point errors that can otherwise produce domain errors when the dot product is computed near the boundary.

The result of arccos is in radians; it is converted to degrees by multiplication with one hundred and eighty divided by π. The pipeline expresses all reported angles in degrees because the biomechanical thresholds in the sports-medicine literature are conventionally specified in degrees and because degrees are more intuitive for end users to interpret. Figure 3.7 illustrates the geometry of the three-point computation, showing the two vectors BA and BC and the interior angle θ formed between them at vertex B.

Figure 3.7: Three-point cosine-rule geometry for joint angle computation. Source: Author.

The following angles are computed during each detected jump: the left-knee angle formed by the left hip, left knee, and left ankle; the right-knee angle formed by the right hip, right knee, and right ankle; and the trunk angle formed by the shoulder midpoint, hip midpoint, and knee midpoint. The trunk angle, which is the most diagnostic single quantity for the anterior-trunk-lean fault, is

computed against the midpoint reference rather than against either side individually because it is intended to capture overall torso inclination rather than asymmetric body posture.

3.8 Biomechanical Fault Detection

The fault-detection stage evaluates five distinct biomechanical fault conditions for each detected jump, using the joint angles computed in the previous stage and the landmark positions retained in the per-frame snapshot. The five conditions are defined as follows.

Left knee valgus is detected when the left knee angle deviates from the ideal one-hundred-and-eighty-degree alignment by more than fifteen degrees, that is, when the absolute value of one hundred and eighty minus the left knee angle exceeds fifteen. Right knee valgus is detected analogously using the right knee angle. The fifteen-degree threshold corresponds to the conventional clinical threshold for biomechanically significant knee valgus reported in the sports-medicine literature.

Anterior trunk lean is detected when the trunk angle (defined in Section 3.7 as the angle formed by the shoulder midpoint, hip midpoint, and knee midpoint) falls below one hundred and fifty degrees, which corresponds to forward inclination of the torso by more than thirty degrees from the vertical reference. Below this threshold, load shifts away from the gluteal complex onto the lower lumbar spine, which is associated with elevated injury risk.

Left knee-over-toe and right knee-over-toe are detected when the corresponding knee landmark's normalised X-coordinate exceeds the corresponding ankle landmark's X-coordinate by more than one tenth of a frame width. The normalised coordinates produced by BlazePose are already in the range [0, 1], so the threshold is applied directly without further scaling. The condition is evaluated separately for the left and right sides because asymmetric knee-over-toe displacement is clinically meaningful and is missed by formulations that average the two sides.

Table 3.3 summarises the five fault conditions, their detection rules, and the point deduction associated with each.

Fault Condition	Detection Rule	Deduction

Left knee valgus	$\lvert 180° - \theta_left_knee \rvert > 15°$	−2 points
Right knee valgus	$\lvert 180° - \theta_right_knee \rvert > 15°$	−2 points
Anterior trunk lean	$\theta_trunk < 150°$ (≡ lean > 30°)	−2 points
Left knee-over-toe	$x_left_knee - x_left_ankle > 0.10$	−2 points
Right knee-over-toe	$x_right_knee - x_right_ankle > 0.10$	−2 points

Table 3.3: Biomechanical fault thresholds and associated point deductions.

Each fault condition is evaluated at the frame within the jump window at which the trigger-point Y-coordinate is at its minimum, which corresponds to the apex of the jump in image space. Evaluating at the apex rather than at, for example, every frame within the window has two advantages: it produces a single, deterministic per-jump fault classification rather than a distribution requiring further aggregation, and it focuses evaluation on the moment at which biomechanical loading is most diagnostic. Figure 3.8 visualises the five fault conditions overlaid on a representative apex-frame skeletal pose.

Figure 3.8: Five biomechanical fault conditions evaluated during each detected jump. Source: Author.

3.9 Quality Scoring Mechanism

Each completed repetition is assigned a quality score on a zero-to-ten scale through a deterministic deduction-based scheme. Every repetition starts with a base score of ten, and each detected fault condition results in a two-point deduction. The score is therefore computed as ten minus twice the number of detected faults, with a floor of zero applied to handle the edge case of more than five fault conditions being detected (which is not possible under the current five-condition design but is guarded against for forward compatibility):

$$score = max(0, 10 - 2 \times n_faults)$$

This scoring scheme is deliberately simple and interpretable. An athlete who completes a repetition with no fault conditions detected receives the full score of ten; an athlete who triggers one fault receives eight; two faults yields six; three yields four; four yields two; and five yields zero. The two-point deduction per fault reflects the design decision that no single fault should dominate the

score and that the per-fault cost should be uniform, given that all five fault conditions are biomechanically significant and clinically grounded.

Alternative weighting schemes were considered during the design phase, including assigning higher deductions to knee-valgus conditions on the basis that anterior-cruciate-ligament injury risk is particularly sensitive to knee valgus. The uniform two-point scheme was ultimately preferred because it preserves interpretability and because the available evidence for differential weighting in a single-camera, normalised-coordinate context is not sufficient to justify a more complex scheme. This decision is revisited in the future-work section of Chapter 5.

3.10 Data Persistence with SQLite

Each completed repetition is persisted to a local SQLite database. SQLite was selected over alternative storage backends for three reasons. First, it is a file-based, server-less embedded database, meaning that no separate database service needs to be running for the application to function, which is consistent with the project's commitment to local-only processing. Second, it provides full ACID-compliant transactional semantics, which guarantees that a partially-written repetition record cannot leave the database in an inconsistent state if the application crashes mid-write. Third, it has excellent Python tooling through the standard library sqlite3 module, eliminating any external dependency.

Table 3.4 documents the schema of the repetitions table, which is the central persisted artefact produced by this component of the pipeline.

Column	Type	Description
rep_id	INTEGER PRIMARY KEY	Auto-incrementing repetition identifier
session_id	TEXT	Foreign key linking to the sessions table
start_frame	INTEGER	First frame of the active jump window
end_frame	INTEGER	Last frame of the active jump window
sensitivity_mode	TEXT	low, medium, or high

score	INTEGER	Quality score in [0, 10]
fault_left_valgus	INTEGER	1 if left knee valgus detected, else 0
fault_right_valgus	INTEGER	1 if right knee valgus detected, else 0
fault_trunk_lean	INTEGER	1 if anterior trunk lean detected, else 0
fault_left_kot	INTEGER	1 if left knee-over-toe detected, else 0
fault_right_kot	INTEGER	1 if right knee-over-toe detected, else 0
snapshot_json	TEXT	JSON-serialised landmark snapshot at apex frame
created_at	TIMESTAMP	Insertion timestamp

Table 3.4: SQLite schema for the repetitions table.

Figure 3.9 illustrates the end-to-end data flow from a single video frame through to a persisted repetition record consumed by the downstream recommendation module developed by another team member. Each arrow in the figure corresponds to a documented inter-stage interface.

Figure 3.9: SQLite schema and end-to-end data flow from frame to recommendation module. Source: Author.

Persistence is performed at the moment a jump's end_frame is recorded, immediately after fault detection and scoring have completed. This timing ensures that the record persisted is complete and that the downstream modules see a fully-populated row when they query the database. Bulk-batch persistence at session end was considered and rejected on the basis that it would introduce a window during which a crash would lose all of a session's data.

3.11 Data Cleaning and Numerical Integrity

Before any feature engineering or regression-based forecasting is performed, the persisted repetition records pass through a data-cleaning layer whose responsibility is to ensure that no numerical pathology contaminates the downstream computations. The cleaning layer applies three categories of guard, each addressing a specific class of failure that arose during development testing.

The first category is safe-division guards. The most common derived feature in the pipeline is the per-session repetition rate, expressed as repetitions per minute. For an empty session in which no jumps were detected, naively dividing by the session duration would still produce a valid zero, but for a session whose duration is itself zero (which can occur in pathological cases such as a session that was started and immediately terminated), naively dividing would raise a ZeroDivisionError or produce an inf value. The cleaning layer therefore wraps every division by a session-derived quantity in a guard that returns zero when the denominator is zero or near-zero.

The second category is non-finite-value filtering. Even in well-behaved sessions, the chain of computations from raw landmarks through smoothing, normalisation, and feature engineering occasionally produces a NaN or an inf when a transient pose-estimation dropout coincides with a denominator near zero. Such non-finite values are mathematically permissible but are catastrophic when fed to a regression model: a single NaN in the training matrix corrupts the entire fit. The cleaning layer applies np.isfinite to every feature before it is admitted to the regression input set, replacing any non-finite value with zero and logging the replacement for forensic review.

The third category is value clamping. Each engineered feature has a documented valid range, derived from its definition. The form-quality feature, computed as the per-session mean of the per-repetition zero-to-ten score, is clamped to the range [0, 10]. The normalised-speed feature, computed as the per-session repetition rate divided by a fifty-rep-per-minute baseline, is clamped to the range [0, 1] for the regression input but the original unclamped value is also retained in a separate column for diagnostic purposes. The volume feature, expressed as a fraction of a one-hundred-rep target, is similarly clamped to [0, 1].

Together, these three categories of guard ensure that the regression model is fed only well-formed, finite, in-range features. The cleaning layer is implemented as a set of pure functions whose inputs and outputs can be unit-tested in isolation, and a small unit-test suite covering the three categories of guard is run as part of the project's continuous integration.

3.12 Feature Engineering for Performance Forecasting

The feature engineering stage transforms the raw repetition records into a small, interpretable set of per-session features suitable for input to the linear regression model described in Section 3.13. Three features are computed: normalised speed, normalised volume, and form quality.

Normalised speed is defined as the per-session repetition rate, expressed in repetitions per minute, divided by a fixed baseline of fifty repetitions per minute. The fifty-rep-per-minute baseline was selected as a representative pace for typical squat-jump and counter-movement-jump training sets, derived from the mean pace observed across the volunteer dataset. The resulting normalised speed feature is dimensionless, equals one for an athlete training at exactly fifty repetitions per minute, and is clamped to the range [0, 1] for input to the regression model. Values above one are retained in the unclamped diagnostic column for forensic review.

Normalised volume is defined as the total number of repetitions per session divided by a target of one hundred repetitions. The target was selected as a representative session-length figure for typical training programmes, again derived from the volunteer dataset. The resulting normalised volume feature is similarly dimensionless and clamped to [0, 1].

Form quality is defined as the mean per-repetition score across the session, divided by the maximum possible score of ten. The resulting form-quality feature is dimensionless and naturally lies in the range [0, 1] without explicit clamping (although the clamping is still applied as a defensive guard).

Table 3.5 summarises the three engineered features, their definitions, their valid ranges, and the baseline values used for normalisation.

Feature	Definition	Valid Range	Baseline
Normalised speed	(reps / minute) / 50	[0, 1]	50 reps/min
Normalised volume	session_reps / 100	[0, 1]	100 reps
Form quality	mean(score) / 10	[0, 1]	10 (perfect)

Table 3.5: Engineered features feeding the regression model.

These three features were chosen for their interpretability and for their orthogonality. Each captures a distinct dimension of athletic performance: speed reflects training pace, volume reflects

training intensity, and form quality reflects movement competence. The deliberate decision to keep the feature set small reflects the small-data regime characteristic of single-athlete training records: with only a few sessions of training data per athlete, a high-dimensional feature set would dramatically increase the risk of overfitting without offering a corresponding accuracy benefit.

3.13 Linear Regression for Longitudinal Forecasting

The forecasting stage trains a linear regression model whose target is the next-session form-quality feature and whose inputs are the current-session and prior-session feature vectors. The model is fit using ordinary least squares as implemented by scikit-learn's LinearRegression class. No regularisation is applied in the present implementation because the small feature set and the small per-athlete sample size make overfitting through coefficient inflation a less salient concern than overfitting through feature explosion, which is already prevented by the deliberate feature-set design.

Figure 3.10 illustrates the complete pipeline from raw repetition records through cleaning, feature engineering, model fit, and prediction. Each step in the figure corresponds to a documented module in the implementation.

Figure 3.10: Linear regression feature pipeline from raw repetitions to forecast output. Source: Author.

The model is re-fit at the end of every session using all sessions accumulated so far for the current athlete. While more sophisticated online or incremental fitting strategies were considered, the per-session re-fit cost is in the millisecond range for typical athlete sample sizes (under a hundred sessions per athlete), and the simpler re-fit-from-scratch implementation has the additional advantage that the model is always trained against the most current data distribution without any need to manage stale state.

3.14 Commercialization Aspects of the Product

The video-pipeline component, while described in this dissertation as part of the broader NeuroMotion system, has commercial value as a stand-alone offering targeted at school-level and recreational athletic programmes. The component requires only a standard webcam and a consumer-grade laptop or desktop, which positions it well for adoption in environments where

dedicated hardware budgets are limited but where coaches and athletes nevertheless want access to objective performance tracking.

The proposed commercialisation model follows a tiered software-as-a-service framework. A free tier provides single-athlete tracking with the most recent thirty days of session history retained, sufficient to demonstrate value to potential paying users without exposing the more bandwidth-intensive longitudinal-trend features. A standard tier provides multi-athlete tracking suitable for school-level coaches, with full session-history retention and access to the regression-based forecasting features. An institutional tier provides multi-team deployment, fleet-wide dashboards, and white-label customisation suitable for sporting academies and university athletics programmes.

Table 3.6 documents the complete technology stack used in the implementation of this component.

Layer	Technology
Backend Language	Python 3.10+
Frontend Framework	Streamlit
Pose Estimation	MediaPipe BlazePose
Computer Vision	OpenCV (cv2)
Numerical Computing	NumPy
Machine Learning	scikit-learn
Data Visualisation	Plotly, Pandas
Persistence	SQLite (stdlib sqlite3)
Logging	Python stdlib logging

Table 3.6: Technology stack used in the video-pipeline component.

3.15 Testing and Implementation

The testing methodology applied to this component encompassed three distinct activities: algorithmic validation against expert-annotated ground truth, performance benchmarking under

representative workloads, and end-to-end integration testing with the downstream modules contributed by other team members.

Algorithmic validation evaluated jump-detection accuracy and form-quality-score agreement against ground-truth labels produced by three independent annotators reviewing the same forty-five test sessions. Inter-annotator agreement was ninety-six point eight percent, providing a reliable basis for treating the consensus labels as ground truth. Performance benchmarking measured per-frame latency across a workload of one thousand frames captured at thirty frames per second, with median, ninetieth-percentile, and ninety-fifth-percentile latencies all reported in Chapter 4.

Integration testing ensured that the SQLite records produced by this component were consumable without modification by the recommendation, voice-feedback, and posture-correction modules. A small integration-test suite, run as part of the project's continuous integration, exercises the full ingestion-to-persistence pipeline against a fixture video and verifies that the resulting database state matches a recorded golden snapshot.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Pipeline Performance Results

The video-pipeline component was evaluated across forty-five test sessions performed by twelve volunteer athletes who were not part of the development cohort, ensuring that the reported results reflect generalisation to unseen athletes rather than memorisation of development data. Each test session lasted between three and twelve minutes and contained between eight and forty-two jump repetitions, for a total test set of six hundred and seventeen individual jumps spanning all three sensitivity modes.

Table 4.1 presents the overall performance evaluation results across nine key metrics that together characterise the effectiveness of the pipeline for practical deployment.

Metric	Value
Jump-Detection Accuracy (overall)	93.2%
Form-Quality Score Frame-Level Agreement	94.1%
Median Per-Frame Latency (CPU)	78ms
95th-Percentile Per-Frame Latency (CPU)	112ms
Calibration Phase Duration	3.3 seconds (100 frames @ 30 FPS)
Calibration Acceptance Rate (first attempt)	92.4%
Database Write Latency (per repetition)	< 5ms
Sustained Session Duration without Crash	Up to 60 minutes (max tested)
Regression R^2 (next-session form quality)	0.81

Table 4.1: Overall pipeline performance evaluation results.

The 93.2% jump-detection accuracy figure was computed against a ground-truth set in which each jump in each test session was independently labelled by three annotators reviewing the source video at half-speed. A detected jump was considered correct if its temporal overlap with a ground-truth jump exceeded fifty percent of the ground-truth jump's duration; precision and recall were

both above ninety percent, with the headline accuracy figure representing the F1-style harmonic mean. The remaining 6.8% error breaks down approximately evenly between false positives, where the threshold-crossing logic registered a jump that the annotators did not, and false negatives, where genuine shallow jumps fell below the medium-sensitivity threshold.

Figure 4.1 visualises the distribution of per-frame latencies across the test workload of one thousand frames. The distribution is sharply concentrated around the median of seventy-eight milliseconds with a long right tail driven by occasional garbage-collection pauses and operating-system scheduling jitter. The 95th-percentile latency of one hundred and twelve milliseconds slightly exceeds the sub-100ms target set in the second research objective; however, the median and ninetieth-percentile latencies remain comfortably within the target, and the user-perceived experience during testing was consistently smooth.

Figure 4.1: Per-frame latency distribution across the test workload. Source: Author.

The 92.4% calibration acceptance rate on first attempt indicates that the calibration window is well-tuned for typical user behaviour. Of the 7.6% of sessions in which the first calibration attempt was rejected because of excessive trigger-point motion during the calibration window, all were successfully calibrated on the second attempt, indicating that the rejection logic correctly identified contaminated baselines without becoming overly restrictive.

4.2 Pose Estimation and Smoothing Effectiveness

The effectiveness of the EMA smoothing stage was quantified by comparing the standard deviation of the trigger-point Y-coordinate during static standing windows before and after smoothing. Across the forty-five test sessions, the mean pre-smoothing standard deviation during static windows was 4.7 pixels (in the original 720-pixel frame height); the mean post-smoothing standard deviation was 1.1 pixels, representing a 76.6% reduction in jitter. This level of jitter suppression is sufficient to make the threshold-crossing logic robust against single-frame outliers without introducing perceptible lag during real motion.

BlazePose's reported sub-100ms inference time was independently confirmed during testing, with the median observed inference time of sixty-eight milliseconds matching the figure reported in the model's published documentation. The remaining ten milliseconds of median per-frame latency,

bringing the total to seventy-eight milliseconds, is consumed by the surrounding pipeline stages: ingestion, smoothing, jump detection, and (when triggered) joint angle computation, fault detection, and persistence.

A small number of frames during testing produced null pose-estimation results, typically when the athlete partially left the frame during a particularly explosive jump. The pipeline's null-handling logic, described in Section 3.3, correctly carried forward the previous smoothed values for one frame in 100% of these cases without producing spurious jump events.

4.3 Jump Detection Accuracy

Per-mode jump-detection accuracy provides a more nuanced view of the system's behaviour than the overall figure reported in Table 4.1. Detection accuracy varies systematically with the selected sensitivity mode, reflecting the different trigger-point landmarks and threshold offsets used by each mode.

Table 4.2 presents the breakdown of detection accuracy by sensitivity mode.

Sensitivity Mode	Trigger Point	Sessions Tested	Detection Accuracy
Low	Ankle midpoint	15	91.4%
Medium	Knee midpoint	20	94.7%
High	Torso centroid	10	92.5%

Table 4.2: Jump-detection accuracy by sensitivity mode.

Medium sensitivity, which is the default, achieves the highest detection accuracy at 94.7%, reflecting the design intent: knee-midpoint displacement is the most diagnostic single signal for typical squat-jump and counter-movement-jump exercises, and the corresponding offset of approximately six percent of frame height is well-tuned to the displacement range characteristic of these exercises. Low sensitivity, which uses the ankle midpoint, accumulates more false negatives because some athletes' ankle landmarks are partially occluded by clothing or footwear during deep squats. High sensitivity, which uses the torso centroid, occasionally produces false

positives during arm-swing-heavy movements where the shoulders contribute more vertical motion than the legs, slightly degrading its overall accuracy.

Figure 4.2 visualises the per-mode accuracy along with a breakdown of false-positive versus false-negative errors, providing a more diagnostic view of the failure modes of each mode. The figure makes clear that the low-sensitivity mode is dominated by false negatives while the high-sensitivity mode is dominated by false positives, consistent with the qualitative analysis above.

Figure 4.2: Jump-detection accuracy by sensitivity mode and exercise variant. Source: Author.

4.4 Form Quality Score Validation

The form-quality scoring mechanism was validated by comparing per-repetition scores produced by the pipeline against scores assigned by the same three expert annotators who labelled the jump events. The annotators used the same five-fault deduction scheme as the pipeline, scoring each repetition by visually identifying which of the five fault conditions were present in the apex frame of the jump.

Frame-level agreement between the pipeline's fault classifications and the consensus annotator classifications, computed as the fraction of (repetition, fault-condition) pairs on which the pipeline and the annotators agreed, was 94.1%. Per-fault breakdown reveals that knee-valgus detection achieved 95.8% agreement on the left side and 96.1% on the right, anterior-trunk-lean detection achieved 92.7% agreement, and knee-over-toe detection achieved 92.0% agreement on the left and 93.9% on the right. The slightly lower agreement for trunk lean and knee-over-toe is attributable to the threshold-boundary cases where the measured angle or displacement is within a degree or two of the threshold and minor pose-estimation noise can flip the classification.

Aggregating to per-repetition scores, the mean absolute error between the pipeline's score and the consensus annotator score was 0.42 points on the zero-to-ten scale, with 88% of repetitions agreeing within one point. The remaining disagreements were almost entirely concentrated in repetitions exhibiting boundary-case fault classifications, and inspection of the disagreements showed that in roughly two-thirds of the cases the pipeline's classification was at least defensible if not preferable to the annotators'. This level of agreement is sufficient to make the per-repetition scores actionable for both the user-facing display and for the downstream forecasting model.

4.5 Performance Forecasting Results

The linear regression model trained on the engineered features (normalised speed, normalised volume, and form quality) was evaluated against a held-out forecast set in which the most recent session of each athlete was withheld during training and used as the prediction target. Across the twelve test athletes, the model achieved a coefficient of determination R-squared of 0.81 on the held-out forecast set, mean absolute error of 0.067 on the [0, 1]-scale form-quality target (corresponding to approximately 0.67 points on the zero-to-ten scale), and root-mean-squared error of 0.084.

Table 4.3 presents the regression evaluation metrics, including the standardised regression coefficients for each input feature.

Metric	Value
Coefficient of Determination (R^2)	0.81
Mean Absolute Error (form quality)	0.067 (≈0.67 points / 10)
Root Mean Squared Error	0.084
Standardised Coefficient: Normalised Speed	0.18
Standardised Coefficient: Normalised Volume	0.24
Standardised Coefficient: Form Quality	0.61
Held-Out Athletes	12
Held-Out Sessions	12 (one per athlete)

Table 4.3: Linear regression evaluation metrics on the held-out forecast set.

The standardised coefficients are particularly informative. Form quality has the largest standardised coefficient of 0.61, indicating that an athlete's current-session form-quality score is the strongest predictor of their next-session form-quality score, which is the expected and desirable behaviour. Normalised volume has the second-largest coefficient at 0.24, indicating that higher training volume is associated with modest improvement in next-session form quality. Normalised speed has the smallest coefficient at 0.18, suggesting that pace is the least diagnostic of the three

features for the form-quality forecasting task. Figure 4.3 visualises the predicted versus observed form-quality scores across the held-out forecast set, showing close alignment along the y-equals-x line.

Figure 4.3: Predicted versus observed form-quality scores for the linear regression model. Source: Author.

These results comfortably exceed the R-squared target of 0.70 set in the fifth research objective and validate the design decision to use a small, interpretable feature set with a simple linear model. More sophisticated alternatives were considered during development but were not pursued because the present configuration already satisfies the project's requirements without sacrificing interpretability.

4.6 Comparison with Existing Approaches

Comparing the present pipeline against alternative approaches for video-based jump analysis highlights its unique combination of capabilities. Table 4.4 provides a feature-by-feature comparison across four representative alternative approaches: bare BlazePose integrations without an analysis layer, commercial fitness applications with proprietary analysis logic, motion-capture laboratories, and the present NeuroMotion video-pipeline component.

Feature	Bare BlazePose	Commercial Fitness Apps	Motion Capture Lab	This Pipeline
Camera-only	Yes	Yes	No	Yes
Real-time (<100ms median)	Yes	Mostly	No	Yes
Per-jump fault detection	No	Limited	Yes	Yes
Sensitivity-mode configurability	No	No	Limited	Yes
Open SQLite persistence	No	No (proprietary)	Vendor-specific	Yes

Longitudinal forecasting	No	Limited	Possible	Yes
Affordable (<$1000 hardware)	Yes	Yes	No	Yes

Table 4.4: Comparison with existing video-pipeline approaches.

Bare BlazePose integrations expose landmarks but provide no analysis layer, leaving the user to build everything from scratch. Commercial fitness applications include analysis but typically as proprietary, opaque logic with no schema-level access to per-jump records and no open longitudinal forecasting. Motion-capture laboratories provide gold-standard accuracy but at prohibitive cost and inaccessibility. The present pipeline occupies a distinct point in this design space: open implementation, accessible hardware, real-time analysis, structured persistence, and longitudinal forecasting in a single coherent stack.

4.7 Research Findings

The research findings produced by this component can be organised into three categories: pipeline-level findings, smoothing-and-detection findings, and forecasting findings. The pipeline-level findings demonstrate that an end-to-end ingestion-to-forecasting stack for jump analysis is feasible on consumer hardware, with median per-frame latency below 100ms and sustained session uptime up to at least one hour. This validates the central claim of the dissertation: that the integration of well-understood individual building blocks into a coherent pipeline constitutes a substantive contribution distinct from merely cataloguing the building blocks themselves.

The smoothing-and-detection findings establish that an exponential moving average filter with α equal to 0.7 produces the best practical balance of jitter suppression and motion responsiveness for the present application, and that a one-hundred-frame calibration window with sensitivity-mode-based threshold derivation produces detection accuracy above ninety-one percent across all three modes. The mode-specific accuracy results, with medium sensitivity achieving the highest accuracy of ninety-four point seven percent, are consistent with the design intent and provide a defensible default for new users.

The forecasting findings establish that a three-feature linear regression model trained on cleaned, clamped per-session features can produce next-session form-quality forecasts with R-squared above 0.80, comfortably exceeding the target of 0.70 set in the fifth research objective. The standardised coefficient analysis identifies form quality itself as the dominant predictor of future form quality, with volume and speed playing modest secondary roles.

4.8 Discussion

The results of this evaluation demonstrate that the five research objectives have been substantively achieved. The dual-mode video ingestion layer operates with sub-twenty-millisecond per-frame latency on both live and recorded inputs. The MediaPipe BlazePose integration delivers stable thirty-three-landmark predictions at sub-100ms median latency. The calibration-aware jump-detection algorithm correctly segments jumps with overall accuracy of 93.2% across all three sensitivity modes. The biomechanical fault-detection layer achieves 94.1% frame-level agreement with expert annotations, exceeding the ninety-percent target. The SQLite-backed persistence layer and three-feature linear regression model produce next-session form-quality forecasts with R-squared of 0.81, comfortably exceeding the 0.70 target.

Several design decisions warrant explicit discussion. The choice of α equal to 0.7 for EMA smoothing was determined empirically and is reported here as a tuned hyperparameter rather than as a theoretical optimum. Different values may be optimal in deployment contexts that differ substantively from the volunteer dataset, particularly contexts with very different ambient frame rates or with substantially more motion blur. The choice of the one-hundred-frame calibration window was similarly empirical and may benefit from further tuning in deployments where session start-up time is more or less constrained than in the present testing context.

The three-mode sensitivity scheme was a deliberate departure from the single-threshold approach common in published examples, and the per-mode accuracy results validate the design. However, the three modes are presented to users as discrete choices, and an interesting future direction would be to derive a continuous sensitivity parameter that adapts to the athlete's observed jump amplitude during the calibration phase. This would shift the calibration burden away from the user and produce a more robustly tuned threshold without requiring explicit mode selection.

Several limitations of the present pipeline warrant acknowledgment. First, the pipeline operates with monocular camera input and produces two-dimensional analysis: the BlazePose Z-coordinate is not used in the active analysis path, which means that movements with significant forward-backward motion relative to the camera plane are not fully evaluated. Stereo-camera or depth-sensor integration would address this limitation but at the cost of requiring additional hardware. Second, the SQLite persistence layer assumes single-writer semantics, which is appropriate for the single-user-per-device deployment model but would require migration to a server-based database for multi-user institutional deployments. Third, the linear regression forecasting model is necessarily limited by the small per-athlete sample size and would benefit from augmentation with priors derived from a larger cross-athlete dataset, an extension that is left to future work.

4.9 Summary of My Individual Contribution

My individual contribution to the NeuroMotion project is the video-pipeline, data-processing, and jump-analysis component reported in this dissertation. This component constitutes the backbone of the system on which the recommendation, voice-feedback, and posture-correction modules contributed by other team members ultimately depend, since it is responsible for transforming raw video frames into the structured per-repetition records that those modules consume.

Specifically, I designed and implemented the dual-mode OpenCV-based video ingestion layer described in Section 3.2, the MediaPipe BlazePose integration described in Section 3.3, the EMA smoothing stage described in Section 3.4, and the calibration-aware jump-detection logic with three-mode sensitivity described in Sections 3.5 and 3.6. I derived and tuned the smoothing coefficient α equal to 0.7, the one-hundred-frame calibration window length, and the three sensitivity-mode offsets through empirical evaluation on the volunteer dataset.

I designed and implemented the three-point cosine-rule joint-angle computation described in Section 3.7, the five-condition biomechanical fault-detection layer described in Section 3.8, and the deterministic deduction-based scoring scheme described in Section 3.9. I designed the SQLite schema described in Section 3.10, including the explicit fault-flag columns and the JSON-serialised landmark snapshot column, in coordination with the team members who consume the persisted records.

I designed and implemented the data-cleaning layer described in Section 3.11, including the safe-division guards, the np.isfinite filtering, and the value-clamping logic. I designed the three-feature feature-engineering pipeline described in Section 3.12, including the fifty-rep-per-minute speed baseline and the one-hundred-rep volume target, and I trained and evaluated the linear regression forecasting model described in Section 3.13. I authored the unit-test suite covering the cleaning layer and the integration-test suite exercising the full ingestion-to-persistence pipeline against fixture videos.

CHAPTER 5: CONCLUSION

5.1 Summary of the Research

This dissertation presented the video-pipeline, data-processing, and jump-analysis component of the NeuroMotion system, a real-time biomechanical exercise-correction platform that demonstrates the feasibility of providing accessible, structured, longitudinally-aware jump analysis using only a standard webcam and a consumer-grade computing device. The component achieves 93.2% jump-detection accuracy and 94.1% form-quality-score agreement with expert annotations while maintaining sub-100ms median per-frame latency, all on hardware that costs an order of magnitude less than the motion-capture installations that have historically been required for joint-level athletic analysis.

The pipeline is composed of nine sequential stages that together transform a raw video frame into a persisted, scored, and forecastable repetition record: ingestion, pose estimation, smoothing, calibration, jump detection, joint-angle computation, fault detection, scoring, and persistence. A tenth stage operating over the persisted records performs feature engineering and linear-regression-based forecasting of next-session form quality. Each stage was designed with explicit fail-isolation, observability, and integration boundaries that cooperate with the modules contributed by other members of the team.

The contribution of this work is fundamentally an engineering contribution rather than a novelty contribution. The individual building blocks (BlazePose pose estimation, EMA smoothing, threshold-based event detection, cosine-rule angle computation, SQLite persistence, and linear regression) are all well-established. What is novel is their integration into a coherent, deployable, tuned, and validated pipeline tailored for jump biomechanics specifically. The dissertation argues, and the evaluation results in Chapter 4 substantiate, that this integration is itself a substantive research contribution that fills a measurable gap in the existing tooling landscape.

5.2 Achievements and Contributions

The five primary achievements of this component, mapped directly to the five research objectives stated in Section 1.4, are as follows. First, a dual-mode OpenCV-based video ingestion layer that

accepts both live webcam capture and pre-recorded video file upload through a unified frame-iterator interface, with measured per-frame ingestion latency below twenty milliseconds. Second, a MediaPipe BlazePose integration that delivers stable thirty-three-landmark predictions at sub-100ms median end-to-end latency on consumer CPUs. Third, a calibration-aware, body-agnostic jump-detection algorithm with three configurable sensitivity modes (ankle, knee, and torso) achieving 93.2% overall detection accuracy across the test set. Fourth, a five-condition biomechanical fault-detection layer based on three-point cosine-rule joint-angle computation, with 94.1% frame-level agreement with expert annotations and a deterministic zero-to-ten quality scoring mechanism. Fifth, a SQLite-backed persistence layer with explicit data-cleaning safeguards and a three-feature linear regression forecasting model achieving R-squared of 0.81 on the held-out next-session forecast set.

Beyond these primary achievements, several secondary contributions warrant noting. The empirical derivation of α equal to 0.7 as the EMA smoothing coefficient, the one-hundred-frame calibration window, and the three sensitivity-mode offsets are all reported with sufficient detail to be reproducible by future researchers working on similar problems. The SQLite schema is documented in full in Appendix B and is suitable for adoption by other groups building comparable pipelines. The three-feature forecasting feature set is similarly documented in Section 3.12 and Appendix C. These artefacts collectively lower the barrier to entry for future work in pose-based jump analysis.

5.3 Limitations

Several limitations of the present component warrant explicit acknowledgment. The first is the monocular-camera constraint inherent in the use of a single webcam: the pipeline operates in two dimensions and does not fully evaluate movements with significant forward-backward motion relative to the camera plane. Athletes performing explosive jumps with substantial forward translation may be incorrectly classified by the knee-over-toe fault rule, which is sensitive only to the in-plane projection. Mitigation through stereo cameras or depth sensors is technically feasible but conflicts with the project's commitment to standard webcam hardware.

The second limitation concerns the small per-athlete sample size that constrains the linear regression forecasting model. With only a handful of sessions per athlete in the typical deployment

scenario, the regression coefficients are necessarily fit on limited data and may produce wider prediction intervals than would be ideal. A future extension that augments per-athlete fits with cross-athlete priors, perhaps through hierarchical Bayesian regression or simple coefficient regularisation toward population means, would address this limitation directly.

The third limitation concerns the uniform two-point deduction scheme used in the quality scoring mechanism. While uniform deductions preserve interpretability, they do not reflect the differential injury risk associated with different fault conditions. A more nuanced scoring scheme that assigns higher deductions to knee-valgus conditions on the basis of anterior-cruciate-ligament injury risk would be biomechanically defensible but would complicate user-facing communication of how scores are computed.

The fourth limitation concerns the SQLite persistence layer's single-writer semantics, which is appropriate for the present single-user-per-device deployment model but would require migration to a server-based database such as PostgreSQL or MySQL for multi-user institutional deployments. The schema design itself is portable and would not require substantive changes during migration, but the surrounding application code would require modification to handle connection pooling and concurrent access semantics.

The fifth limitation concerns the static nature of the smoothing coefficient and calibration window. Both α and the calibration length were tuned for the typical thirty-frames-per-second consumer webcam capture rate, and substantial deviation from this rate would call for re-tuning. An adaptive variant that derives both parameters from observed frame-rate and motion characteristics during the calibration phase would be a useful future extension.

5.4 Future Work and Recommendations

Several directions for future work are suggested by the present component, each addressing one or more of the limitations described above. The first direction is depth-aware extension of the fault-detection logic. While the present pipeline does not actively use the BlazePose Z-coordinate, the coordinate is retained in the per-frame snapshot and could be incorporated into the fault rules to produce three-dimensional fault evaluations. The most immediate target is the knee-over-toe rule,

which would benefit substantially from a depth-aware formulation that distinguishes genuine forward knee translation from in-plane projection artefacts.

The second direction is the introduction of cross-athlete priors into the forecasting model. By fitting a hierarchical model that shares strength across athletes for the regression coefficients while preserving per-athlete intercepts, the system could produce more reliable forecasts during the early sessions when per-athlete data is most limited. This would be particularly valuable for newly-onboarded users for whom the current per-athlete fit has insufficient data to converge.

The third direction is the expansion of the fault-detection set beyond the present five conditions. Additional conditions such as asymmetric leg loading (detected through differential ankle Y-coordinates at landing), insufficient hip flexion at the bottom of the counter-movement, and excessive arm-swing contribution (detected through wrist landmark velocity at take-off) would provide a more complete biomechanical evaluation. Each new condition would require its own threshold derivation through expert consultation and empirical tuning, but the present three-point cosine-rule infrastructure is already in place to support them.

The fourth direction is adaptive calibration. The present pipeline uses a fixed one-hundred-frame calibration window and fixed mode-specific threshold offsets. An adaptive variant could observe the athlete's jump amplitude during the first few uncalibrated jumps and derive a per-athlete continuous sensitivity parameter that replaces the discrete three-mode scheme, producing a more robustly tuned threshold without requiring explicit mode selection by the user.

The fifth direction is institutional deployment with concurrent multi-user support. This would require migration of the SQLite persistence layer to a server-based database, the introduction of authentication and per-athlete data isolation, and the design of fleet-wide dashboards suitable for school-level or university-level coaches managing multiple athletes simultaneously. The application architecture is modular enough to support this migration without substantive changes to the core analysis logic.

In conclusion, this research has demonstrated that an end-to-end, real-time, accessible video-pipeline-and-jump-analysis component is achievable on consumer hardware with accuracy levels and forecasting performance suitable for practical training applications. The component closes a measurable gap between expensive motion-capture laboratories and opaque commercial fitness

applications and provides a foundation upon which the broader NeuroMotion system, and by extension future research in pose-based athletic analysis, can build.

REFERENCE LIST

[1] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," in Proc. CVPR Workshop Comput. Vis. Sports, 2020, pp. 1-8.

[2] Y. Liao, J. Vakanski, and M. Xian, "A deep learning framework for assessing physical rehabilitation exercises," IEEE Trans. Neural Syst. Rehabil. Eng., vol. 28, no. 2, pp. 468-477, Feb. 2020.

[3] Z. Cao, G. Hidalgo, T. Simon, S. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 1, pp. 172-186, Jan. 2021.

[4] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120-126, Nov. 2000.

[5] S. Bridgeman, M. Volino, J. Guillemaut, and A. Hilton, "Multi-person 3D pose estimation and tracking in sports," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW), 2019, pp. 2861-2870.

[6] H. Fang, S. Xie, Y. Tai, and C. Lu, "RMPE: Regional multi-person pose estimation," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), Venice, Italy, 2017, pp. 2334-2343.

[7] C. Wang, A. Bochkovskiy, and H. Liao, "YOLOv7: Trainable bag of freebies sets new state-of-the-art for real-time object detectors," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Vancouver, Canada, 2023, pp. 7464-7475.

[8] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub, Jan. 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[9] A. Ignatov, "Real-time human activity recognition from accelerometer data using convolutional neural networks," Appl. Soft Comput., vol. 62, pp. 915-922, Jan. 2018.

[10] T. E. Hewett, G. D. Myer, and K. R. Ford, "Anterior cruciate ligament injuries in female athletes: Part 1, mechanisms and risk factors," Am. J. Sports Med., vol. 34, no. 2, pp. 299-311, Feb. 2006.

[11] T. Lin et al., "Microsoft COCO: Common objects in context," in Proc. Eur. Conf. Comput. Vis. (ECCV), Zurich, Switzerland, 2014, pp. 740-755.

[12] C. Chen, R. Jafari, and N. Kehtarnavaz, "A real-time human action recognition system using depth and inertial sensor fusion," IEEE Sensors J., vol. 16, no. 3, pp. 773-781, Feb. 2016.

[13] M. A. O'Reilly, D. F. Whelan, T. E. Ward, E. Delahunt, and B. M. Caulfield, "Technology in strength and conditioning: Assessing body weight squat performance," J. Strength Cond. Res., vol. 31, no. 2, pp. 549-555, Feb. 2017.

[14] T. Dressler, J. Witt, and M. Schmitt, "Comparison of optical motion capture and inertial measurement unit methods for measuring joint angles in athletic movements," Sensors, vol. 21, no. 9, pp. 3159-3169, Apr. 2021.

[15] Pedregosa et al., "Scikit-learn: Machine Learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[16] D. Hipp, "SQLite," 2024. [Online]. Available: https://www.sqlite.org

[17] S. van der Walt, S. C. Colbert, and G. Varoquaux, "The NumPy array: A structure for efficient numerical computation," Comput. Sci. Eng., vol. 13, no. 2, pp. 22-30, Mar. 2011.

APPENDICES

Appendix A: BlazePose Landmark Index Reference Used by the Pipeline

This appendix documents the eight BlazePose landmark indices consumed by the pipeline, their corresponding anatomical landmarks, and their roles in each analysis stage. The full BlazePose model emits thirty-three landmarks, but only the subset documented below is consumed in the active analysis path. Remaining landmarks are retained in the per-frame snapshot for forensic purposes but are not used by the threshold-crossing, joint-angle, or fault-detection logic.

BlazePose Index	Anatomical Landmark	Used By
11	Left shoulder	Trunk-lean angle (upper reference)
12	Right shoulder	Trunk-lean angle (upper reference)
23	Left hip	Trunk-lean vertex; valgus reference
24	Right hip	Trunk-lean vertex; valgus reference
25	Left knee	Knee valgus; knee-over-toe; medium-mode trigger
26	Right knee	Knee valgus; knee-over-toe; medium-mode trigger
27	Left ankle	Knee valgus; knee-over-toe; low-mode trigger
28	Right ankle	Knee valgus; knee-over-toe; low-mode trigger

Appendix B: SQLite Schema Specification and Sample Inserts

This appendix presents the complete SQLite schema used by the persistence layer, expressed in canonical CREATE TABLE syntax. The schema comprises two tables: sessions, which records one row per training session, and repetitions, which records one row per detected jump within a session. The two tables are joined by the session_id foreign key.

Table: sessions	

session_id	TEXT PRIMARY KEY
athlete_id	TEXT NOT NULL
started_at	TIMESTAMP NOT NULL
ended_at	TIMESTAMP
sensitivity_mode	TEXT NOT NULL
calibration_baseline_y	REAL NOT NULL
calibration_threshold_y	REAL NOT NULL
frame_count	INTEGER NOT NULL DEFAULT 0

Table: repetitions	
rep_id	INTEGER PRIMARY KEY AUTOINCREMENT
session_id	TEXT NOT NULL REFERENCES sessions(session_id)
start_frame	INTEGER NOT NULL
end_frame	INTEGER NOT NULL
score	INTEGER NOT NULL CHECK (score BETWEEN 0 AND 10)
fault_left_valgus	INTEGER NOT NULL DEFAULT 0
fault_right_valgus	INTEGER NOT NULL DEFAULT 0
fault_trunk_lean	INTEGER NOT NULL DEFAULT 0
fault_left_kot	INTEGER NOT NULL DEFAULT 0
fault_right_kot	INTEGER NOT NULL DEFAULT 0
snapshot_json	TEXT
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Appendix C: Linear Regression Feature-Set Documentation

This appendix documents the three engineered features fed to the linear regression forecasting model, along with their definitions, valid ranges, baseline values, and the rationale for each baseline.

The normalised speed feature is defined as the per-session repetition rate, expressed in repetitions per minute, divided by a fifty-rep-per-minute baseline. The fifty-rep-per-minute baseline was derived from the mean pace observed across the volunteer dataset and is held constant in the present implementation. Future deployments targeting populations with substantially different typical paces (for example elite plyometric athletes who might routinely train at substantially higher paces) may benefit from an adjustment to this baseline.

The normalised volume feature is defined as the total repetition count for the session divided by a one-hundred-rep target. The one-hundred-rep target similarly reflects the typical session length observed in the volunteer dataset. As with the speed baseline, deployments targeting populations with different typical session lengths may benefit from adjustment.

The form-quality feature is defined as the per-session mean of the per-repetition zero-to-ten score, divided by ten. This feature naturally lies in the range [0, 1] without explicit clamping, although clamping is applied as a defensive guard. The form-quality feature is the largest-magnitude predictor of next-session form quality in the trained regression model, with a standardised coefficient of 0.61.

Appendix D: Pseudocode for the Jump Detection State Machine

This appendix provides pseudocode for the jump-detection state machine described in Section 3.6. The machine maintains a single boolean state variable, in_jump, and emits start_frame and end_frame events as the trigger-point Y-coordinate crosses the calibrated threshold. The pseudocode below assumes that the smoothed trigger-point Y-coordinate has already been computed and is named smoothed_y, and that the calibrated baseline and threshold are named baseline_y and threshold_y respectively. The pseudocode is presented in a Python-flavoured syntax for clarity.

```
in_jump = False
start_frame = None
for frame_idx, smoothed_y in stream:
      if not in_jump and smoothed_y < threshold_y:
            in_jump = True
            start_frame = frame_idx
      elif in_jump and smoothed_y >= baseline_y:
            in_jump = False
            end_frame = frame_idx
            if (end_frame - start_frame) >= MIN_JUMP_FRAMES:
                  emit_jump(start_frame, end_frame)
```

The minimum-jump-frames guard, denoted MIN_JUMP_FRAMES in the pseudocode, is set to five frames in the present implementation. This guard prevents single-frame or two-frame transient threshold crossings (typically caused by residual jitter or by incidental hand or torso movement during the calibration window) from being registered as spurious repetitions.